EF Core: Bulk Operations & Performance

Quick Notes • Practical performance optimization

1. The Core Problem

Calling **SaveChanges()** inside a loop repeatedly runs EF Core's save pipeline and creates avoidable database communication overhead. With 10,000 records, this can mean 10,000 separate save operations.

2. AddRange + One SaveChanges

AddRange() marks many entities as **Added** in the Change Tracker. It does not write to the database. The actual database work happens when **SaveChanges()** is called.

```
context.Students.AddRange(students);
context.SaveChanges();
```

EF Core detects changes, generates the required INSERT commands, executes them, and normally changes the entities' states to **Unchanged** after a successful save.

Important: One SaveChanges call does not mean one SQL statement or exactly one network round trip. EF Core may execute multiple statements in one or more batches, depending on the provider and configuration.

3. BulkInsert

EFCore.BulkExtensions provides specialized bulk operations for large data sets and can use provider-specific bulk mechanisms.

```
context.BulkInsert(students);
```

BulkInsert does not follow the normal SaveChanges/Change Tracker workflow in the same way. Verify requirements such as generated identity values, navigation properties, tracking state, transactions, and provider/version compatibility. It is not automatically the best choice for every workload.

4. Choosing the Right Approach

Approach	Typical use	Guidance
SaveChanges in loop	Repeated individual saves	Avoid for large batches
AddRange + SaveChanges	Grouped CRUD inserts	Good default choice
BulkInsert	Very large data imports	Benchmark for your workload

5. Benchmarking & Practical Notes

• **Stopwatch:** measure elapsed time under the same conditions for each approach.

• **Fresh DbContext:** use a separate context for each test to avoid accumulating tracked entities.

• **Realistic data:** test representative volumes, such as 1,000 and 10,000 records.

• **ToQueryString():** displays SQL for LINQ queries; it does not preview INSERT commands generated by SaveChanges().

• **CanConnect():** checks connectivity; it is not a performance benchmark.

• **Sensitive data logging:** useful during development, but avoid it in production because logged values may be sensitive.

Mental Model

Reduce repeated saves. Use AddRange + SaveChanges for ordinary grouped inserts. Consider BulkInsert for very large workloads, then benchmark and verify the behavior your application needs.